Project - High Level Design

on

Deploy Education App to Kubernetes

DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AMAN VERMA	EN22CS301110
02	ANJALI JAISWAL	EN22CS301132
03	ANKIT PARMAR	EN22CS301135
04	ANSH LAAD	EN22CS301141
05	APOORVA GUPTA	EN22CS301190
06	ARYA BHAWSAR	EN22CS301209

Group Name:12D2

Project Number:DO-24

Industry Mentor Name:

University Mentor Name:Akshay Saxena

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Interfaces

4. Data Design
   4.1 Data Model
   4.2 Data Access Mechanism
   4.3 Data Retention Policies
   4.4 Data Migration

5. State and Session Management

6. Caching

7. Functional Requirements
   7.1 User Authentication & Authorization
   7.2 Infrastructure Configuration Management
   7.3 Provisioning Control C Monitoring
   7.4 Dockerfile (Application Containerization)
   7.5 GitHub Actions (Workflow Configuration)
   7.6 CI/CD Strategy (Continuous Integration C Deployment)

8. Non-Functional Requirements

9. References

10. GitHub

1.Introduction

The **Education App** is an AI-powered educational application designed to generate structured, engaging, and visually rich learning content from a simple topic prompt. It leverages advanced Large Language Models (LLMs) and AI-based image generation to enhance conceptual understanding through both textual explanations and visual representations.

The system is built with modern technologies and follows a complete DevOps lifecycle, including containerization using Docker and automated deployment via CI/CD pipelines on AWS EC2. This ensures scalability, reliability, and real-world production readiness.

The application is accessible through a web interface and supports real-time content generation, making it useful for students, educators, and self-learners.

1.1 Scope of the Document

This document provides a detailed overview of the **design, functionality, and deployment** of the Multimodal Education Creator application.

It covers:

- System architecture and workflow
- Features and functionalities
- Technology stack used
- Deployment process on AWS EC2
- Docker and CI/CD pipeline integration
- Environment configuration

The document is intended to serve as a reference for:

- Developers maintaining or extending the system
- DevOps engineers handling deployment and automation
- Users who want to understand system capabilities

1.2 Intended Audience

This document is intended for the following audience:

- **Developers**
  To understand the system architecture, backend logic, and integration of AI services.
- **DevOps Engineers**
  To manage deployment, CI/CD pipelines, Docker containers, and AWS infrastructure.
- **Educators & Students**
  To understand how the application generates learning content and how it can be used effectively.
- **Project Evaluators / Recruiters**
  To assess the technical depth, real-world implementation, and scalability of the project.

1.3 System Overview

The Multimodal Education Creator is a cloud-deployed AI application that transforms a user-provided topic into structured educational content along with AI-generated visuals.

Core Workflow

1. User inputs a topic via the Streamlit interface
2. The request is sent to the Gemini API for text generation
3. The generated content is processed into structured learning sections
4. A prompt is created for image generation
5. Hugging Face API generates relevant visuals
6. The output (text + images) is displayed in the UI

System Architecture Overview

The system follows a **client-server architecture** deployed on the cloud:

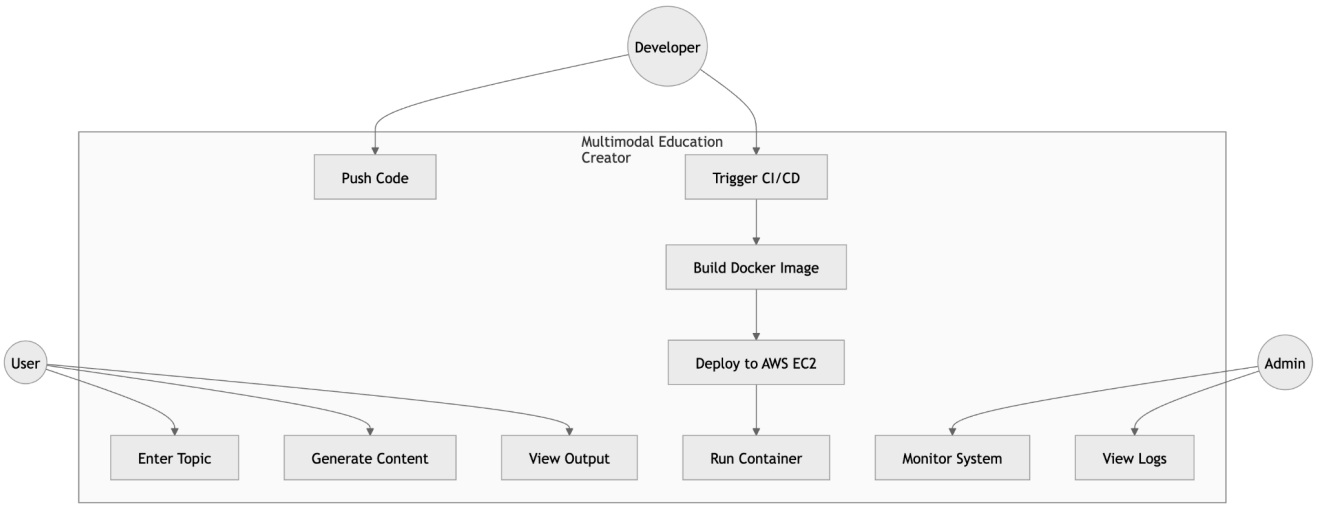
- **Frontend (Client Layer)**:
  Built using Streamlit, provides an interactive user interface
- **Backend (Application Layer)**:
  Developed in Python, handles business logic and API communication
- **AI Services (Processing Layer)**:
  - Gemini API for text generation
  - Hugging Face API for image generation
- **Deployment Layer**:
  - Docker for containerization
  - AWS EC2 for cloud hosting
  - GitHub Actions for CI/CD automation

2. System Design

The system is designed as a **modular, scalable, and cloud-deployable AI application**. It follows a **microservice-inspired architecture**, ensuring a clear separation between the presentation layer, application logic, external AI services, and deployment infrastructure.

The application integrates **text generation and image generation services** to provide a multimodal learning experience. It is containerized using Docker and deployed on a cloud environment, enabling scalability, portability, and ease of maintenance.

Additionally, the system incorporates a **CI/CD pipeline** to automate the build and deployment process, ensuring continuous integration and rapid delivery of updates.



The Use Case Diagram represents the interaction between different actors and the **Multimodal Education Creator system**. It provides a high-level overview of how various users interact with the system and how the system responds to their actions.

The diagram captures both:

- **Application-level interactions**, where the user generates educational content by providing a topic and receiving structured explanations along with AI-generated visuals.
- **DevOps-level interactions**, where the developer manages code updates and triggers the CI/CD pipeline, and the administrator monitors system performance, deployment status, and logs.

This dual representation ensures a **comprehensive system perspective**, covering both **end-user functionality** and **backend operational workflows** such as containerization, deployment, and monitoring.

Overall, the use case diagram helps in understanding the roles, responsibilities, and interactions within the system, making it easier to analyze system behavior at a high level.

Actors Identified

🔹 **User**

- Interacts with the application
- Generates educational content

🔹 **Developer**

- Maintains and updates the system
- Pushes code changes to repository

🔹 **Admin**

- Monitors system performance
- Manages deployment and logs

Use Cases

👤 User Use Cases

- Enter Topic
- Generate Content (Text + Image)
- View Output

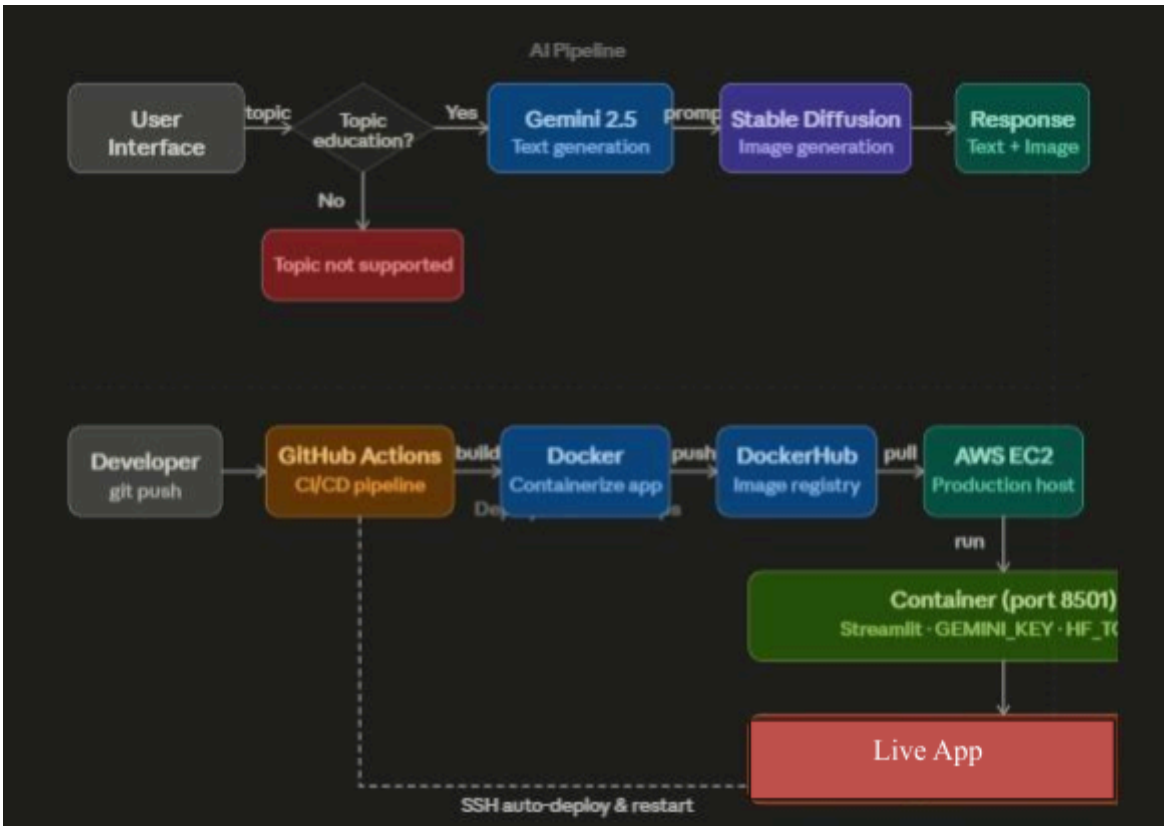
👨‍💻 Developer Use Cases

- Push Code to GitHub
- Trigger CI/CD Pipeline

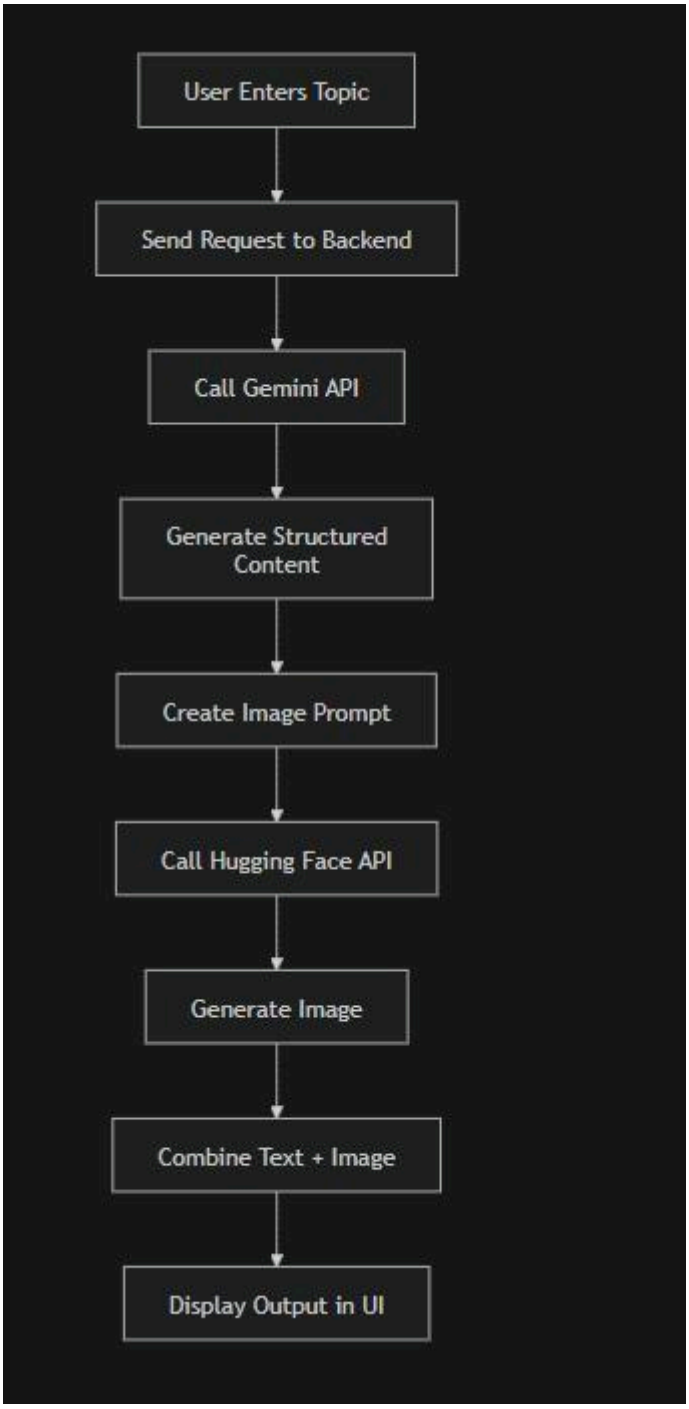
🛠️ Admin Use Cases

- Monitor System
- View Logs
- Manage Deployment

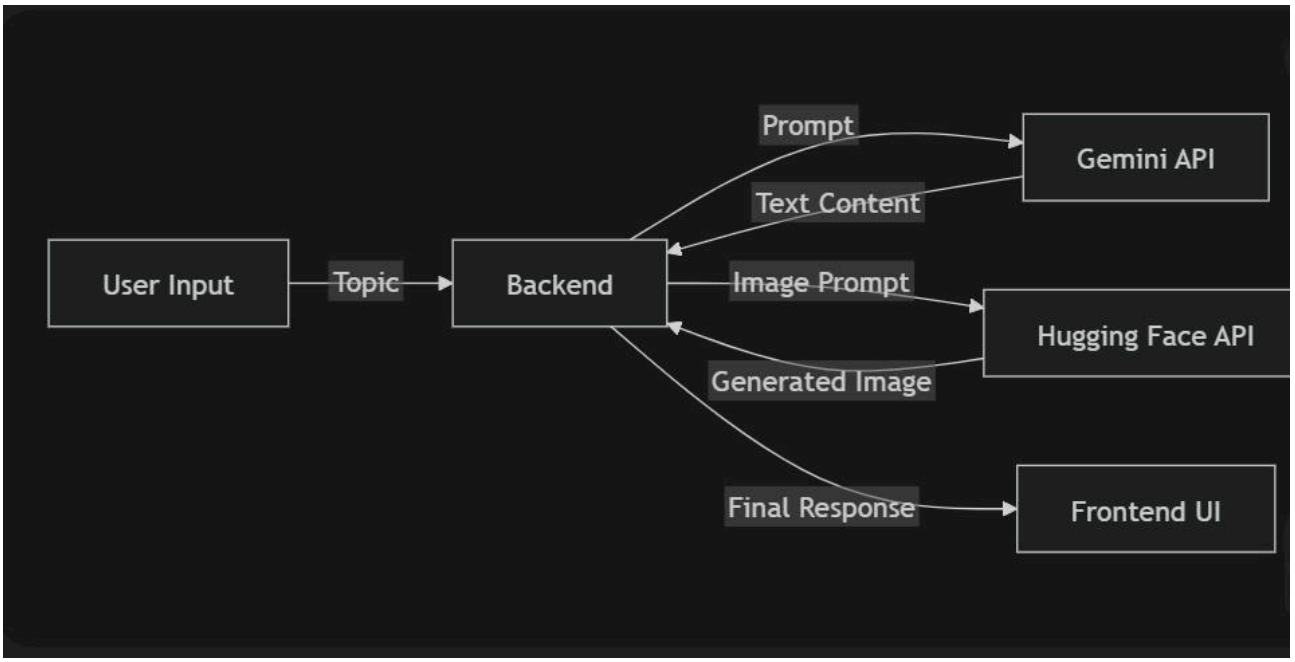
2.1 Application Design



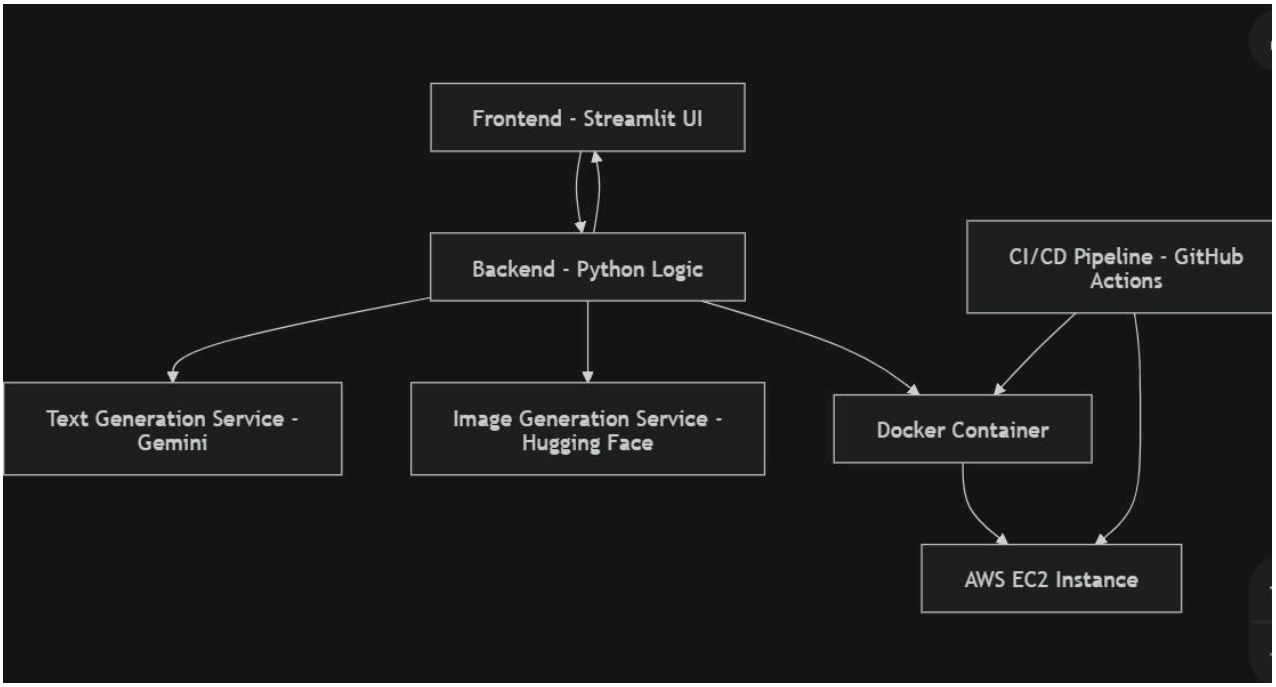
2.2 Process Flow



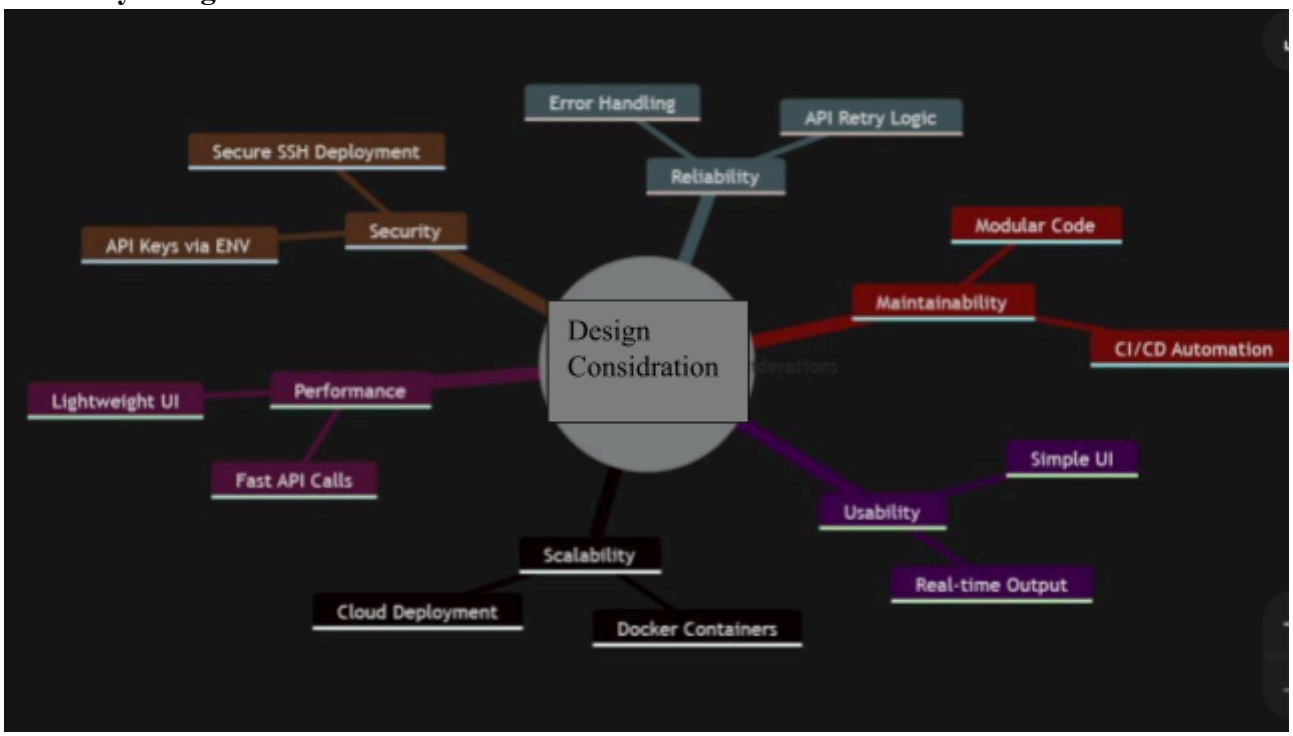
2.3 Information Flow



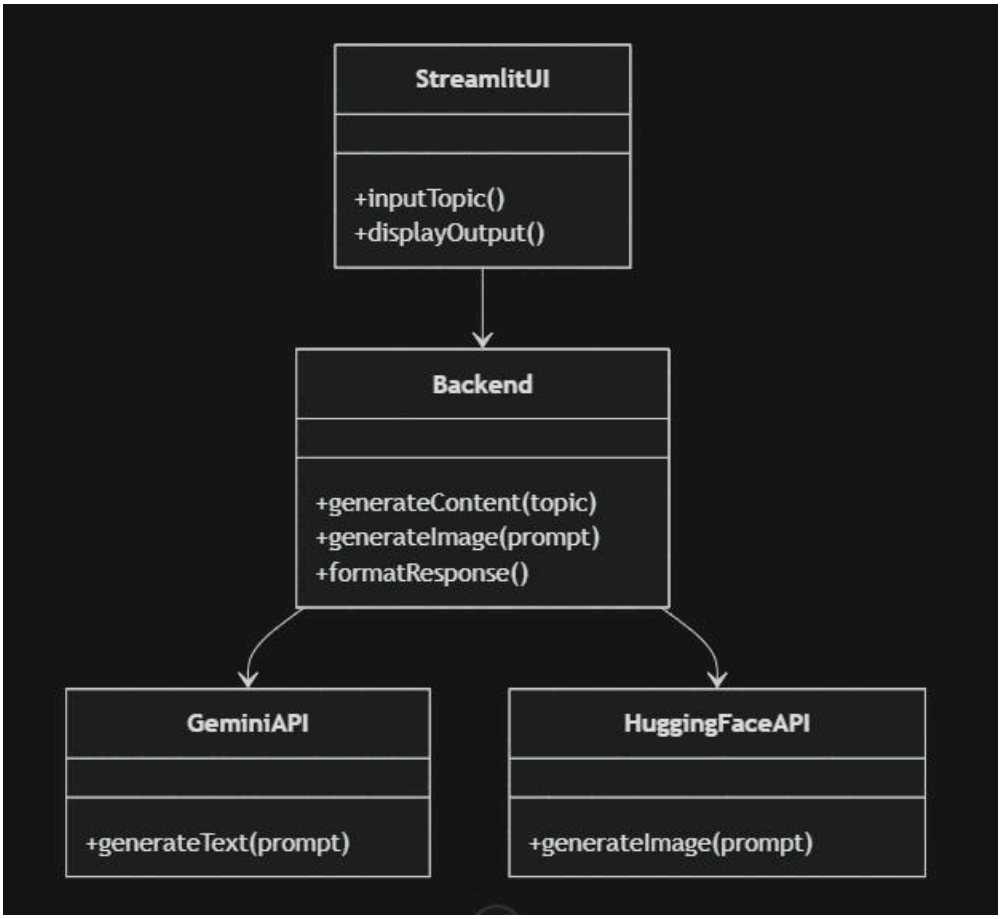
2.4 Components Design



2.5 Key Design Considerations



2.6 API Catalogue.



3.Interface

The system provides a simple and interactive user interface using Streamlit, where users can enter a topic and generate AI-based educational content. The interface displays structured text explanations along with relevant AI-generated images.

The application communicates with external services such as the Gemini API for text generation and Hugging Face API for image generation. Additionally, the system interfaces with Docker and AWS EC2 for deployment, ensuring accessibility through a web browser.

🔷 User Interface (UI)

The user interface is built using **Streamlit**, providing a lightweight and interactive web-based environment.

🔹 **Features:**

- Input field for entering topic
- Button to trigger content generation
- Display area for:
  - Structured explanation
  - Key learning points
  - Generated images

🔷 Infrastructure Interface

The application interacts with deployment infrastructure using containerization and cloud services.

🔹 **Interfaces:**

- Docker → Application containerization
- AWS EC2 → Hosting environment
- GitHub Actions → CI/CD automation

🔷 API Interface

The application communicates with external AI services through APIs.

🔹 **Integrated APIs:**

- **Gemini API** → Text generation
- **Hugging Face API** → Image generation

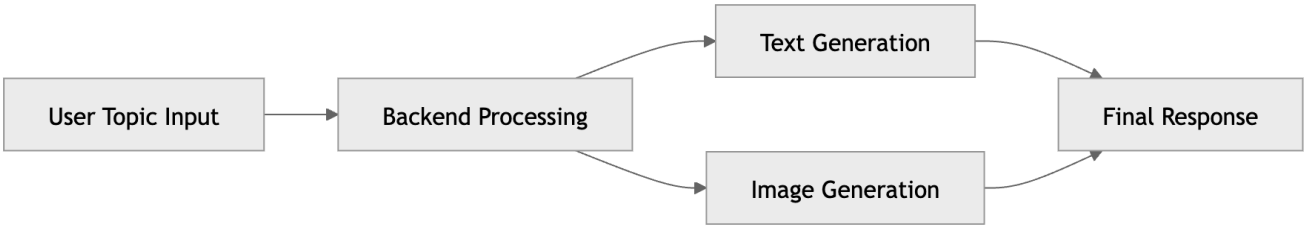
4. Data Design

4.1 Data Model

The system follows a stateless, API-driven data model where no persistent database is used. Data is generated dynamically and flows through different processing layers.

🔷 **Key Data Elements:**

- User Input → Topic (string)
- Text Output → Explanation + Key Points
- Image Output → AI-generated visual



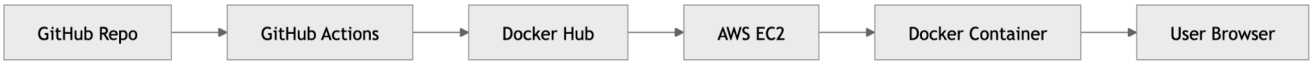
4.2 Data Access Mechanism

🔹 **How data moves:**

- GitHub → CI/CD → Docker Hub → AWS EC2

🔹 **Access Methods:**

- Git (source code)
- Docker pull/push
- SSH (deployment to EC2)
- HTTP (user accessing app)



4.3 Data Retention Policies

- No persistent storage is implemented
- Data exists only during execution (session-based)
- No user tracking or history storage
- API keys are securely stored as environment variables

4.4 Data Migration

Currently, no migration is required due to lack of database.

🔮 Future Scope:

- Add database (MongoDB/PostgreSQL)
- Store generated content
- Enable history tracking

5. State and Session Management

- Managed using **Streamlit session state**
- Maintains:
  - Current user input
  - Generated results

6. Caching

- Currently minimal caching
- Possible Improvements:
  - Cache API responses using Streamlit caching (`@st.cache_data`)

7. Functional Requirements

7.1 User Authentication & Authorization

Current:

- Not implemented

Future:

- Login/Signup system
- Role-based access (Admin/User)

7.2 Infrastructure Configuration Management

🔹 **Description:**

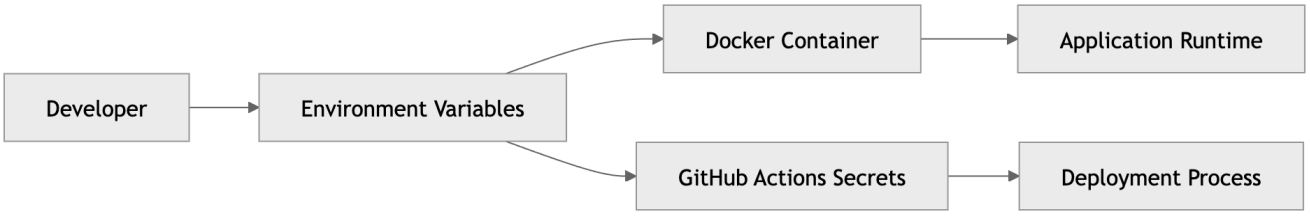
- Manages configuration of applications across environments.

🔹 **Implementation:**

- Environment variables:
  - `GEMINI_API_KEY`
  - `HF_TOKEN`
- Docker environment injection
- GitHub Secrets for CI/CD

🔹 **Benefits:**

- Secure configuration
- Environment independence (dev/prod)



7.3 Provisioning Control & Monitoring

- EC2 instance provisioning
- Docker container lifecycle management
- Basic monitoring via logs

7.4 Dockerfile (Application Containerization)

🔹 **Purpose:**

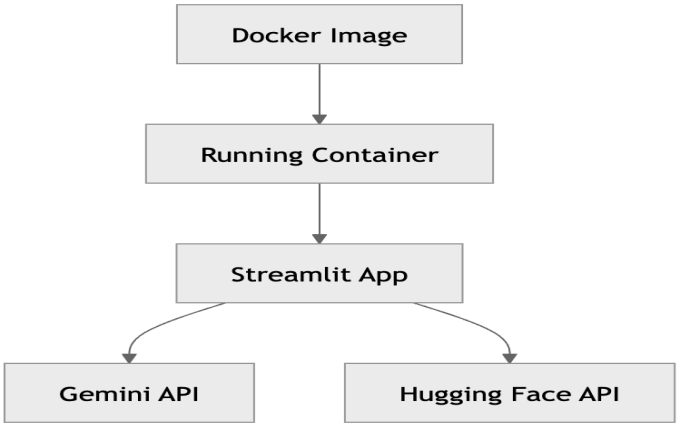
Defines how the application is packaged and run.

🔹 **Key Steps:**

1. Use Python base image
2. Install dependencies
3. Copy source code
4. Run Streamlit app

🔹 **Benefits:**

- Consistency across environments
- Easy deployment

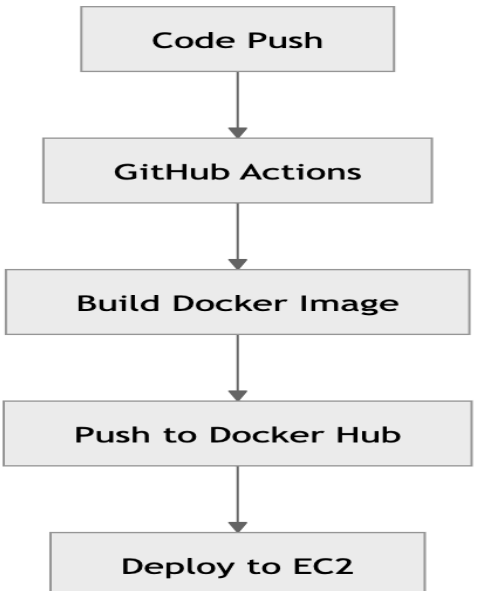


7.5 GitHub Actions (Workflow Configuration)

CI/CD implemented using **GitHub Actions**.

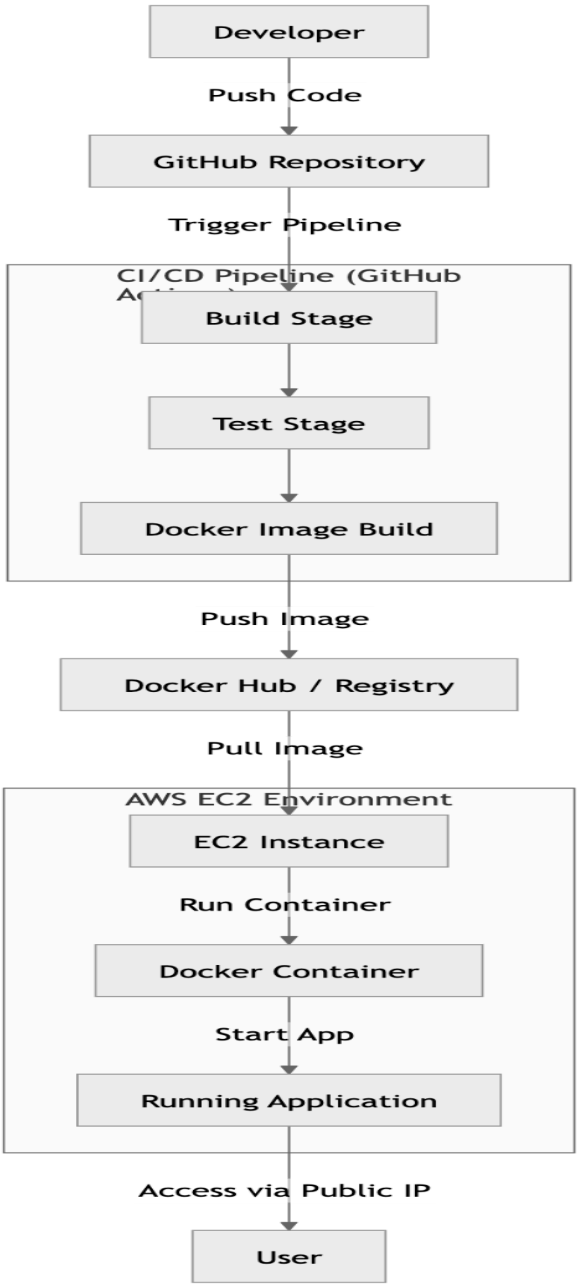
🔄 **Workflow Steps:**

- Trigger on push
- Build Docker image
- Push to DockerHub
- Deploy to EC2



7.6 CI/CD Strategy (Continuous Integration & Deployment)

- **Continuous Integration (CI):**
  - Automated build on every push
  - Ensures code consistency
- **Continuous Deployment (CD):**
  - Auto deployment to AWS EC2
  - Container restart with latest version



8. Non-Functional Requirements

- **Scalability**: Cloud-ready via Amazon Web Services
- **Performance**: Real-time API response
- **Reliability**: Containerized deployment
- **Security**:
  - API keys secured
  - No data persistence
- **Usability**: Simple UI using Streamlit
- **Maintainability**:
  - Modular design
  - CI/CD automation

9.References

Research Papers & Concepts

1. Russell, S., & Norvig, P.
   Artificial Intelligence: A Modern Approach
   Pearson Education
   Fundamental concepts of intelligent agents and AI systems.
2. OpenAI
   GPT Models and Generative AI Documentation
   Concepts of Large Language Models and text generation.
3. Lewis, P., et al.
   Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)
   Introduction to RAG architecture.

DevOps & Deployment

1. Docker Documentation
   https://docs.docker.com/
2. Amazon Web Services – EC2 Documentation
   https://docs.aws.amazon.com/ec2/
3. GitHub – GitHub Actions Documentation
   https://docs.github.com/en/actions

10. GitHub

👉 **https://github.com/nameisankit/education-app**